



SemPDF

Semantic search across a library of PDFs, using a small embedding model that runs locally on the server.

300 level project documentation. Built with Next.js 16, Drizzle ORM and PostgreSQL. Demo login: demo@sempdf.app / demo1234.

- | | |
|--------------------------------|---------------------------------|
| 1. Overview | 6. Running locally |
| 2. How the core logic works | 7. Testing |
| 3. Architecture and data model | 8. Deployment |
| 4. Walkthrough of every screen | 9. 5 minute presentation script |
| 5. Mobile view | 10. Decisions made |

Overview

SemPDF lets a student upload a pile of PDFs (lecture notes, handbooks, research briefs) and search them by meaning instead of by exact words. A search for *best time to plant corn* finds the Maize Farming Guide even though the word corn never appears in it, because the model knows that corn and maize, and plant and sow, mean nearly the same thing.

Core features

- Upload up to 10 PDFs at a time. Text is extracted page by page, split into passages and embedded on the server.
- Semantic search with a local embedding model (all-MiniLM-L6-v2, quantized, 23 MB). No external AI service and no API key.
- Fast: a search over the full demo library takes about 10 to 50 ms. Repeat searches are served from saved results in a few milliseconds.
- Every search is saved to PostgreSQL with its ranked results, and shown in a history page grouped by day.
- Each result shows the document, page number, match strength and the passage itself, and links to the exact passage.
- Email and password accounts. Every query is scoped to the signed-in user.
- YouTube colour theme with light and dark modes, built mobile first with a bottom tab bar on phones.

Sample data

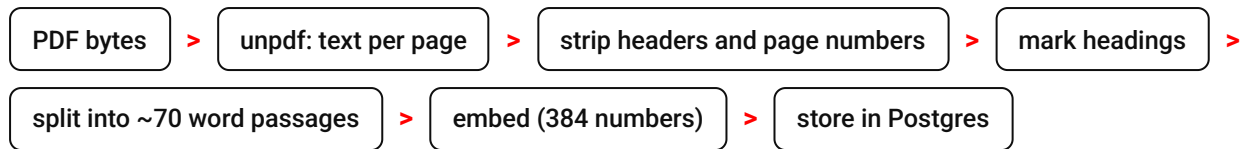
The seed creates a demo student, **Adaeze Okafor**, with **42 PDFs** (40 text documents and 2 scanned ones) across 8 collections: Computer Science, Agriculture, Economics, Health, Law, Engineering, Environment and General. The content is written for a Nigerian university: cassava farming in Ogun State, the Lagos Tenancy Law, CBN monetary policy, malaria prevention, solar sizing for a shop in Enugu, and more. It also creates **73 saved searches** spread over the last 60 days, some run several times. A second user, Tunde Bakare, owns 2 engineering PDFs so you can show that libraries are private.

Problem cases seeded on purpose: two scanned PDFs with no text layer (flagged "No text found"), three searches that match nothing or only weakly (for example "premier league transfer news"), and older searches marked "Will refresh" because PDFs were added after they ran.

All dates are relative to "today". Set `SEMPDF_TODAY=YYYY-MM-DD` to pin today for demos and tests.

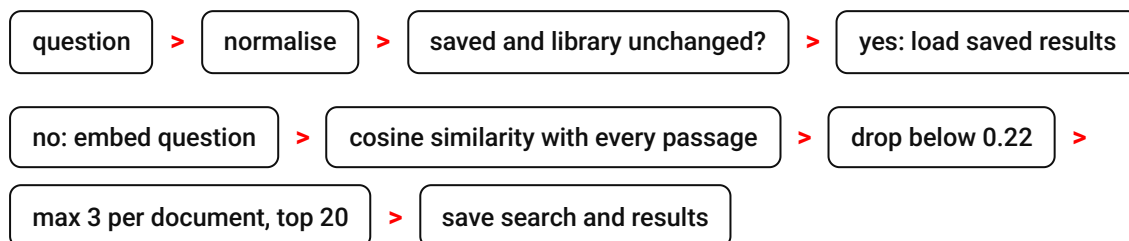
How the core logic works

1. Indexing a PDF



- **Extract.** `unpdf` (a server build of Mozilla pdf.js) returns the text of each page. If no page has text, the document is saved with status `no_text` so the user knows it needs OCR.
- **Clean.** `stripRunningLines` removes a page's first or last line when it repeats on at least 60% of pages (running titles, "Page 3 of 9"). `markHeadings` ends short title lines with a full stop so a heading does not run into the paragraph.
- **Chunk.** `chunkPages` groups whole sentences into passages of up to 70 words, never across a page boundary, and repeats the last sentence of one passage at the start of the next so context is not lost at the cut.
- **Embed.** Each passage goes through all-MiniLM-L6-v2 with mean pooling and normalisation, giving a unit length vector of 384 floats. They are stored as raw bytes (1,536 bytes each) in a `bytea` column.
- The user's `library_version` goes up by one, which tells saved searches they are out of date.

2. Searching



- **Normalise.** Lowercase and collapse spaces, so "Best time" and "best time" are the same saved search.
- **Cache.** `isCacheFresh` compares the search's saved library version with the user's current one. If they match, the saved ranked results are returned without running the model.
- **Rank.** `rankBySimilarity` scores every passage with cosine similarity, which measures the angle between two vectors: 1 means the same direction (same meaning), 0 means unrelated. At most 3 passages per document are kept so one long PDF cannot fill the page.
- **Explain.** `matchStrength` turns the score into Strong (0.55 and up), Good (0.40 and up) or Weak. `hasKeywordMatch` checks whether a passage shares any real word with the question, which powers the "keyword search would have found only X" message and the "Found by meaning" badge.
- **Speed.** The model loads once per server process (about 0.2 s). Each user's vectors are kept in memory keyed by library version, so a fresh search is one model call (about 10 ms) plus a few thousand multiply-adds.

All of these rules are pure functions in `src/lib/` (text, vector, format, today, search-rules, validation) and are covered by unit tests.

Architecture and data model

Layer	What it does	Where
Next.js 16 App Router	Server Components render every page. Server Actions handle sign in, upload, delete and history. Turbopack build.	<code>src/app</code>
Proxy	Redirects signed-out visitors to /signin (Next 16 renamed middleware to proxy).	<code>src/proxy.ts</code>
Auth	bcryptjs hashes, jose HS256 JWT in an HTTP-only, SameSite=Lax cookie. Every page and action re-checks the user in the database.	<code>src/server/auth*.ts</code> , <code>session.ts</code>
Embedding	@huggingface/transformers on onnxruntime-node, model files committed in <code>models/</code> , remote downloads disabled.	<code>src/server/embedder.ts</code>
Ingest and search	PDF parsing, chunking, embedding, ranking and the saved results cache.	<code>src/server/ingest.ts</code> , <code>search.ts</code>
Business rules	Pure functions that take "today" as an argument.	<code>src/lib</code>
Database	PostgreSQL through Drizzle ORM. drizzle-kit writes versioned SQL migrations.	<code>src/db</code> , <code>drizzle/</code>

Tables

Table	Key columns	Notes
users	id, email (unique), name, password_hash, library_version	library_version goes up on every upload and delete.
documents	id, user_id, title, filename, collection, page_count, word_count, chunk_count, status, file (bytea), uploaded_at	The original PDF is stored in the row, so no disk volume is needed.
chunks	id, document_id, user_id, page, position, content, embedding (bytea)	One row per passage. user_id is copied in so search never needs a join to scope by user.
searches	id, user_id, query, normalized_query, embedding, result_count, top_score, duration_ms, run_count, library_version, last_run_at	Unique on (user_id, normalized_query): repeating a search updates the same row.
search_results	search_id, chunk_id, rank, score	The saved ranking, replaced when a stale search runs again.

All foreign keys cascade on delete: deleting a document removes its passages and any saved results that pointed at them. Every query in `src/server/library.ts` and `search.ts` filters on `user_id`.

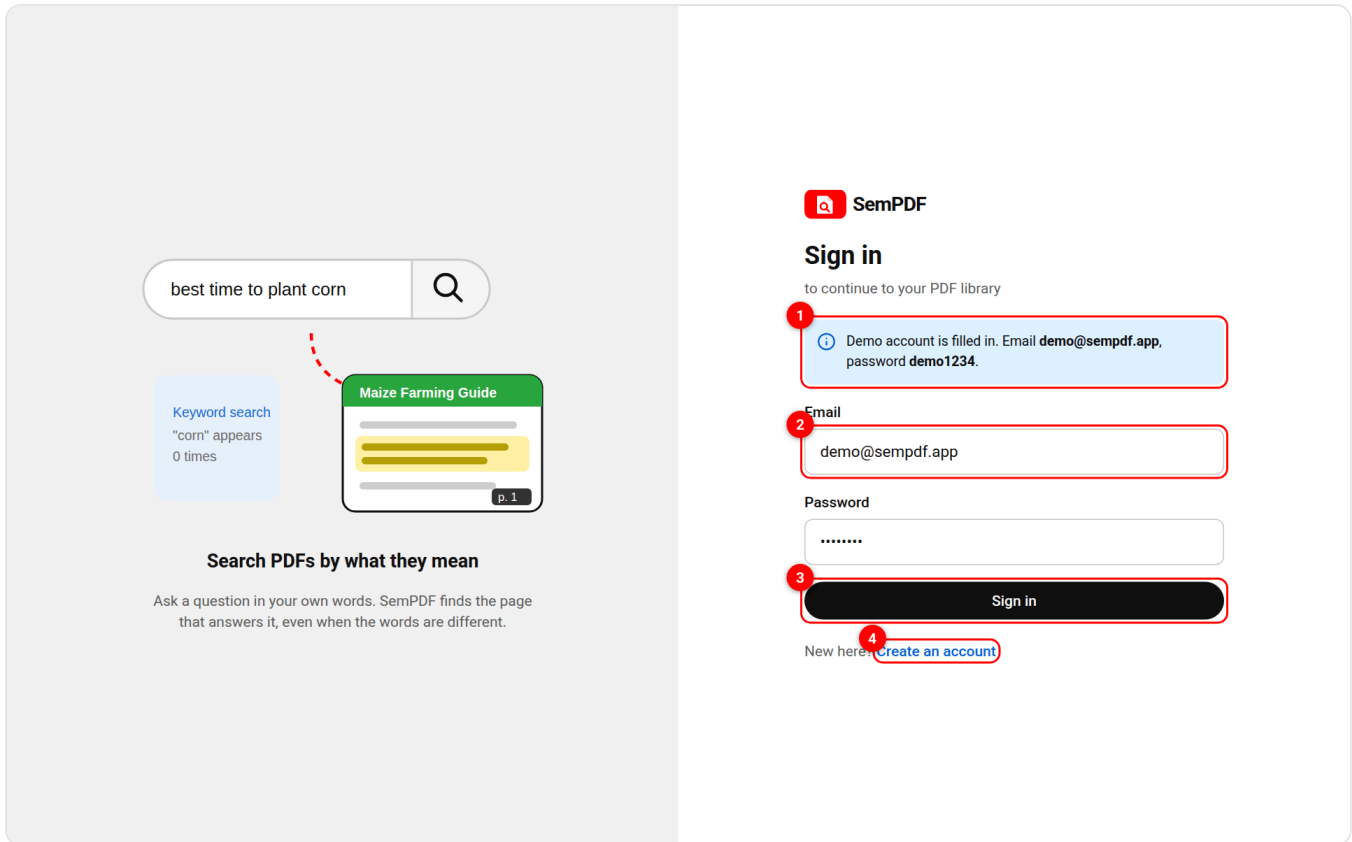
Why no pgvector?

Vectors are compared in Node instead of with a Postgres extension. For a student library (thousands of passages) this is a few milliseconds, it works on any Postgres including the shared server used in deployment, and it keeps the ranking logic in a pure, testable function.

Walkthrough of every screen

Sign in

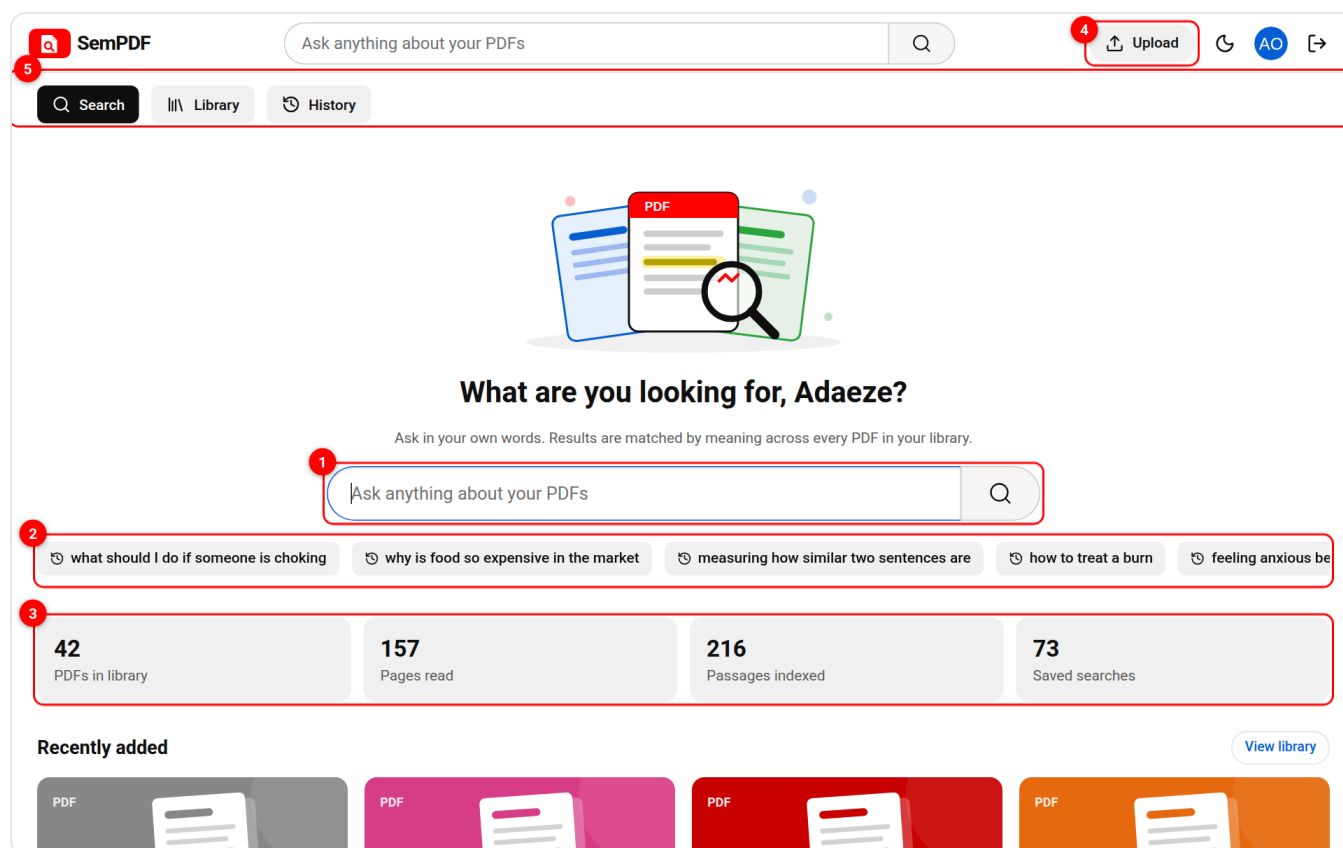
The first screen. The demo account is already filled in so the presenter only clicks one button.



- 1 Demo credentials are shown and pre-filled: demo@sempdf.app / demo1234.
- 2 Email field. Zod validates it on the server and shows the error under the field.
- 3 Sign in sets a signed JWT in an HTTP-only cookie (jose, 7 days).
- 4 New users can create their own account with an empty, private library.

Search home

After sign in. A large YouTube style search bar, suggestion chips and a summary of the library.



Search results

Results for "best time to plant corn". The word corn does not appear in any document, yet the Maize guide ranks first.

SemPDF

best time to plant corn

Upload

AO

Search

Library

History

15 passages from 8 documents

Searched 216 passages in 17 ms

Saved to history

A plain keyword search would have found only 5 of these 15 passages (33%). The rest share the meaning of your question but not its words.

All 15

Agriculture 11

Economics 2

Environment 1

Engineering 1

PDF

MFG

p. 2

5

Strong match

62%

Agriculture

Page 2 of 4

6

eeding the crop. Maize is a heavy feeder, especially of nitrogen. Apply NPK at planting or within two weeks, then top dress with urea at about five to six weeks when the plants are knee high. Place fertiliser in a small hole about five centimetres from the plant and cover it, rather than scattering it on the surface...

PDF

MFG

p. 4

Good match

51%

Agriculture

Page 4 of 4

Found by meaning

Harvest and storage. Harvest when the husks turn brown and the grains are hard. Dry the cobs well before shelling, then dry the grain further to about 13 percent moisture before storage. Poorly dried maize grows moulds that produce aflatoxin, a poison that harms people and livestock. Store grain in hermetic bags or...

PDF

MFG

p. 1

Good match

47%

Agriculture

Page 1 of 4

Kaduna State Agricultural Development Project. Land preparation and planting. Maize needs fertile, well drained soil and good rainfall during the growing season. In the northern guinea savanna, planting usually starts in late May or June once the rains are steady. Plant two seeds per hole at a depth of about five...

PDF

MFG

p. 4

Good match

43%

Agriculture

Page 4 of 4

Soil Fertility and Fertiliser Use

- 1 How many passages matched, how long it took, and whether the result came from the saved cache.
- 2 Comparison with plain keyword search: how many of these results a keyword search would have missed.
- 3 Filter the results by collection.
- 4 Thumbnail with the page number badge. The red bar shows where the page sits in the document.
- 5 Match strength from cosine similarity: strong, good or weak, with a percentage.
- 6 The matching passage. Words shared literally with the question are highlighted.

Document view

Opening a result shows the full text of the document, page by page, with the matched passage highlighted.

- 1 Document title and collection.
- 2 The passage that matched the search, highlighted in blue and scrolled into view.
- 3 Facts about the file: pages, passages, words and size.
- 4 Open the original PDF, find similar documents, or delete it (which also refreshes saved searches).

Library

Every uploaded PDF as a card, like a video grid. 42 sample PDFs across 8 collections are seeded.

The screenshot shows the SemPDF web application interface. At the top, there's a search bar with the text "Ask anything about your PDFs" and a search icon. To the right are buttons for "Upload", a refresh icon, a user profile icon labeled "AO", and a share icon. Below the search bar are tabs for "Search", "Library" (which is active), and "History".

The main section is titled "Your library" and shows a summary: "42 PDFs · 157 pages · 216 passages indexed". Below this is a collection filter bar with chips: "All 42", "Computer Science 8", "Economics 7", "Agriculture 6", "Health 6", "Engineering 4", "Environment 4", "Law 4", and "General 3".

The library displays a grid of PDF cards. Each card has a colored background, a PDF icon, a title, a subtitle, and a "3 pages" badge. The first card, "Photographed Past Questions ECO 301", is highlighted with a red box and a red circle containing the number 1. A red circle containing the number 2 points to the collection filter chips. A red circle containing the number 3 points to a warning icon and text "No text found" below the first card. A red circle containing the number 4 points to the "3 pages" badge on the first card.

The cards shown are:

- Photographed Past Questions ECO 301 (Economics, Not searchable · Today)
- First Aid Quick Reference (Health, 5 passages · Yesterday)
- Introduction to Econometrics (Economics, 3 passages · 2 days ago)
- Electric Motors and Drives (Engineering, 3 passages · 3 days ago)
- Post Harvest Losses in Tomato Supply Chains (Agriculture, 3 pages)
- Oil Spills in the Niger Delta (Environment, 3 pages)
- Cybersecurity Awareness for Students (CS, 4 pages)
- Introduction to Criminal Procedure (Law, 3 pages)

- 1 Totals for the signed-in user only.
- 2 Collection filter chips with counts.
- 3 Problem case: a scanned PDF with no text layer is flagged as not searchable.
- 4 Page count badge, like the duration badge on a video.

Upload

Choose one or more PDFs and a collection. Each file is read, split and embedded on the server, then reported here.

The screenshot shows the SemPDF web interface. At the top is a search bar with the text "Ask anything about your PDFs" and a search icon. To the right are buttons for "Upload", a refresh icon, a user profile icon labeled "AO", and a share icon. Below the search bar are tabs for "Search", "Library", and "History". The main heading is "Upload PDFs" with a subtext: "Up to 10 files at a time, 10 MB each. Text is extracted, split into passages and embedded on this server." The interface is divided into two main sections. The left section contains a file selection area (1) with a dashed border, a file icon, the text "Choose PDF files", "Select one or more .pdf files", and buttons for "Choose Files" and "No file chosen". Below this is a collection selection dropdown (2) labeled "Collection" with "General" selected. At the bottom of this section is a red "Upload and index" button. The right section (3) is titled "What happens next" and shows a success message for a file named "Beekeeping Field Notes" with a green checkmark, stating "2 pages, 2 passages indexed" and providing a link to "Open document". It also shows "Finished in 0.5 s." and a "Search your library" button.

- 1 Pick up to 10 PDFs (10 MB each). Non-PDF files are rejected with an inline error.
- 2 Collection select. It stays uncontrolled so it keeps a valid value after the form resets.
- 3 Per-file outcome: pages read, passages indexed, or the reason it failed.

Search history

Every search is saved to Postgres with its ranked results, grouped by day like YouTube watch history.

The screenshot shows the SemPDF web application's search history page. At the top, there's a navigation bar with 'SemPDF' logo, a search bar, and buttons for 'Upload', 'AO', and a link icon. Below the navigation bar, there are tabs for 'Search', 'Library', and 'History'. The 'History' tab is active. The main section is titled 'Search history' and contains a message: '73 saved searches. Opening one shows its saved results instantly unless your library has changed.' A red box with a '4' callout highlights a 'Clear all history' button. Below this, the history is grouped by date. The 'Today' group is highlighted with a red box and a '1' callout. It contains five search entries, each with a clock icon, a search query, and a 'Will refresh' button. A red box with a '2' callout highlights the search details (results, best score, search count, time, date) for the first entry. A red box with a '3' callout highlights the 'Will refresh' button for the first entry. The 'Yesterday' group contains three search entries, each with a clock icon, a search query, and a 'Will refresh' button.

Search history

73 saved searches. Opening one shows its saved results instantly unless your library has changed.

Today

- what should I do if someone is choking**
7 results best 63% searched once 4 ms Today Will refresh
- why is food so expensive in the market**
20 results best 63% searched 3 times 2 ms Today Will refresh
- best time to plant corn**
15 results best 62% searched 4 times 17 ms Today Will refresh
- measuring how similar two sentences are**
3 results best 45% searched 2 times 2 ms Today Will refresh
- how to treat a burn**
4 results best 59% searched once 14 ms Today Will refresh

Yesterday

- feeling anxious before exams**
5 results best 59% searched 2 times 2 ms Yesterday Will refresh
- regression analysis assumptions**
4 results best 42% searched once 10 ms Yesterday Will refresh
- protecting myself from internet fraud**
6 results best 49% searched 2 times 5 ms Yesterday Will refresh

- 1 Groups: Today, Yesterday, This week, This month, Older.
- 2 Result count, best score, how many times it ran, and how long the search took.
- 3 Will refresh: PDFs were added or removed since, so the next open re-runs the search.
- 4 Clear all history, or remove one search with the bin icon.

Dark theme

The moon icon switches to the YouTube dark palette. The choice is saved in a cookie.

The screenshot shows the SemPDF search interface in dark theme. At the top, the search bar contains the query "can my landlord throw me out without notice". To the right of the search bar is an "Upload" button and a moon icon (labeled 1) for toggling the theme. Below the search bar, there are tabs for "Search", "Library", and "History". A status bar indicates "7 passages from 4 documents" and "Searched 218 passages in 10 ms". A summary bar states: "A plain keyword search would have found only 3 of these 7 passages (43%). The rest share the meaning of your question but not its words." Below this, there are filters for "All 7", "Law 6", and "General 1". The search results are displayed in a list. The first three results are for "Tenancy and Landlord Rights in Lagos" (PDF, TAL, p. 2). The first result has a "Strong match" (59%) and the text: "After the notice expires, the landlord must serve a notice of intention to recover possession, usually seven days, and then go to court. Locking out a tenant, removing the roof or throwing out property without a court order is unlawful." The second result has a "Strong match" (56%) and the text: "Notice to quit. A landlord must serve a proper notice to quit before recovering possession. The length depends on the type of tenancy: one week for a weekly tenancy, one month for a monthly tenancy and six months for a yearly tenancy. After the notice expires, the landlord must serve a notice of intention to recover...". The third result has a "Good match" (41%) and the text: "Always get a written agreement and a receipt for every payment. Under the Lagos State Tenancy Law, it is unlawful for a landlord to demand more than one year's rent in advance from a sitting tenant, or more than one year's rent from a new tenant for a monthly tenancy." The fourth result is for "Law of Contract Study Notes" (PDF, p. 1) with a "Weak match" (38%) and the text: "Found by meaning" (labeled 2).

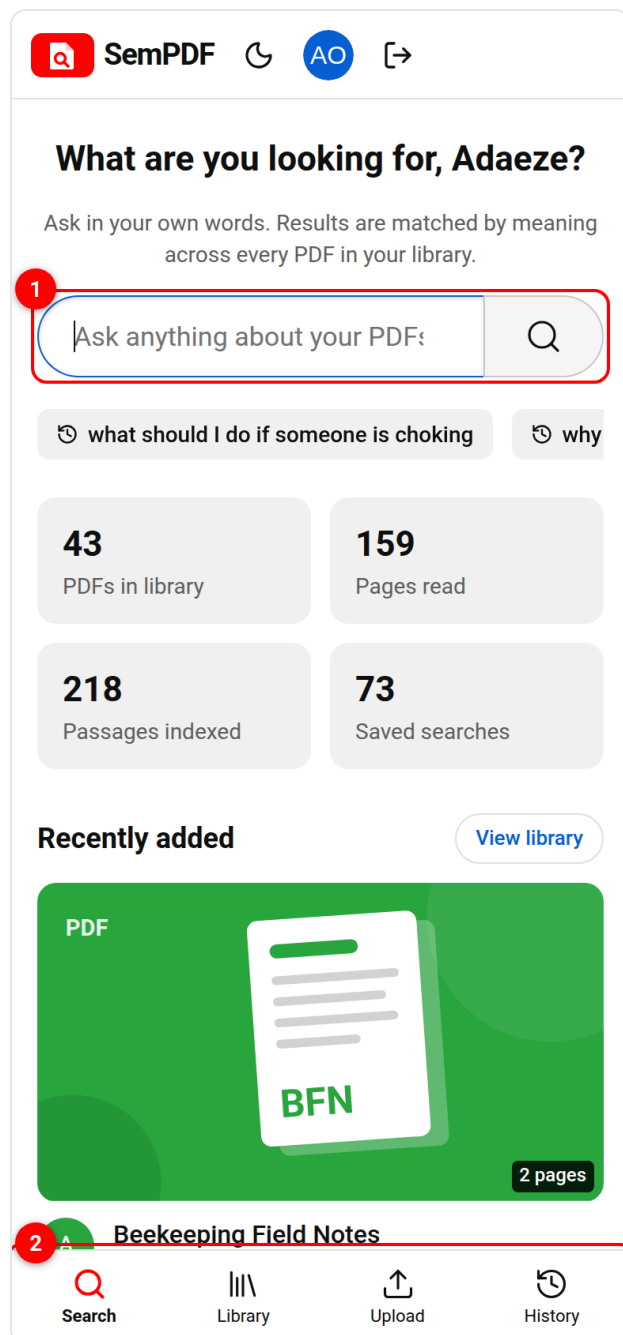
- 1 Theme toggle.
- 2 Found by meaning: this passage shares no keywords with the question.

Mobile view

Below 640 px wide the header search moves into the page, the top navigation becomes a bottom tab bar, and result thumbnails shrink to 120 px. A Playwright test on a Pixel 7 profile checks every tab for sideways overflow.

Mobile home

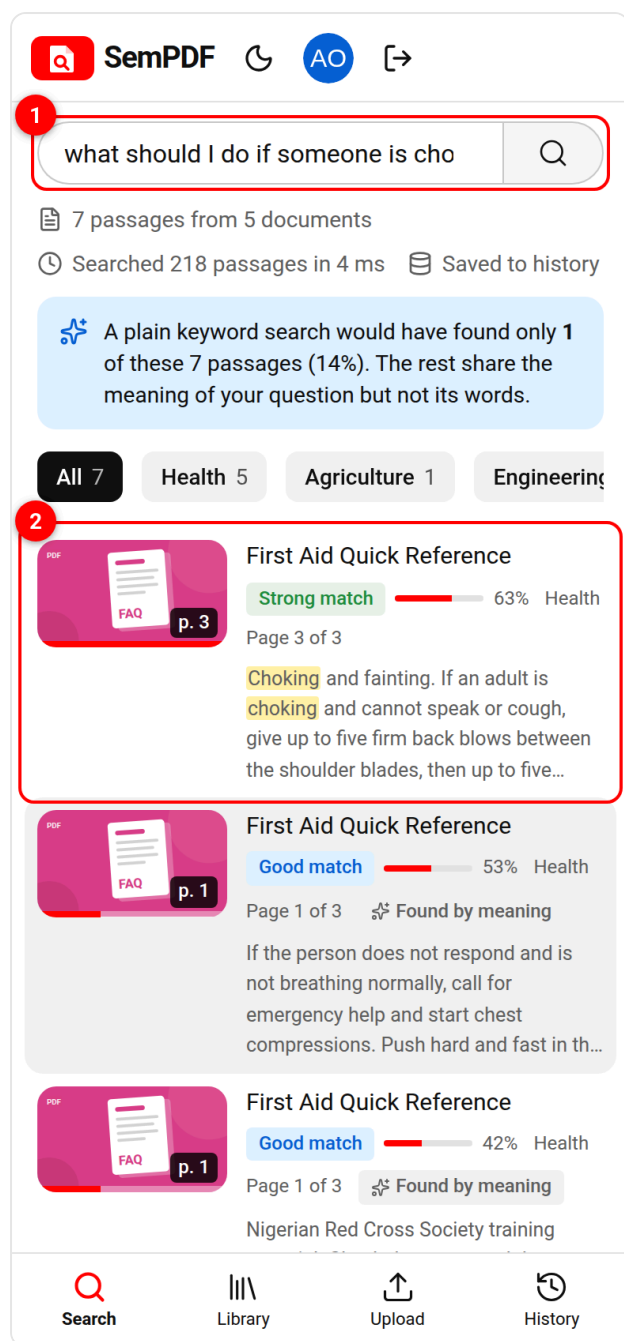
On phones the header shrinks and navigation moves to a bottom tab bar.



- 1 Search bar sized for thumbs.
- 2 Bottom tab bar: Search, Library, Upload, History.

Mobile results

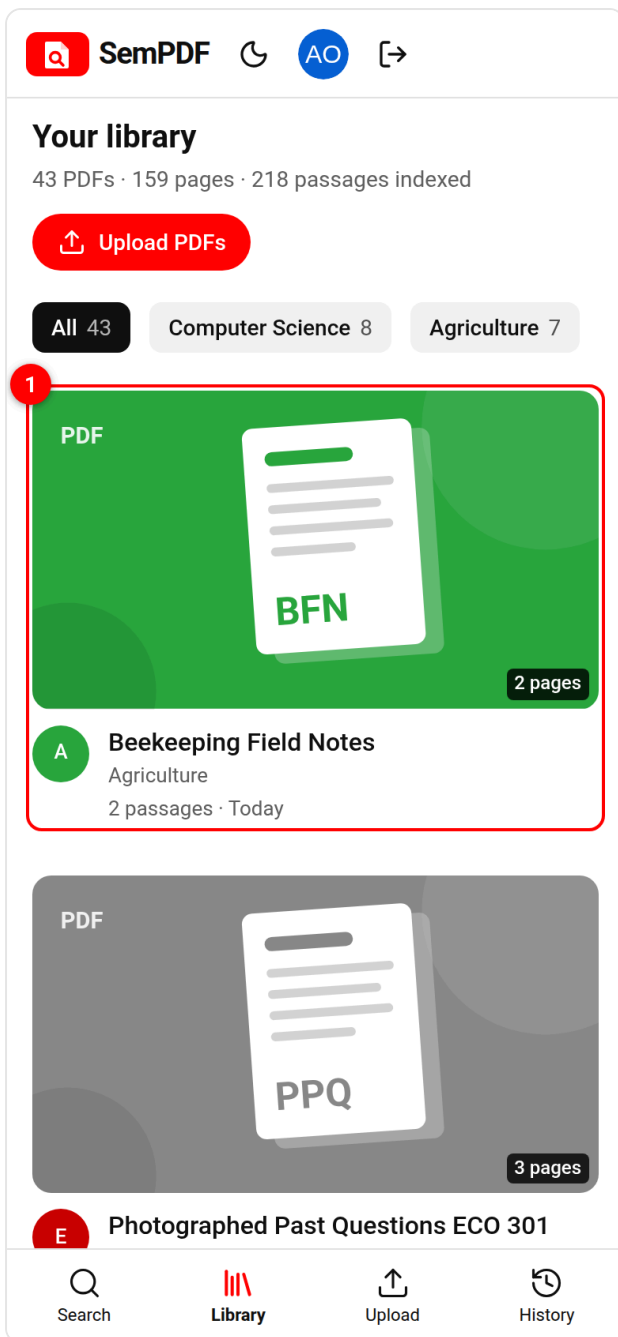
Results stack with a smaller thumbnail. Snippets are clamped to four lines.



- 1 The search bar moves into the page on small screens.
- 2 Top result: First Aid Quick Reference, page 3.

Mobile library

Cards become a single column. Grids use `minmax(0, 1fr)` so nothing overflows sideways.



1 Document card at phone width.

Running locally

Requirements: Node.js 20.9 or newer and PostgreSQL 14 or newer. The embedding model is already in the repository, so no download is needed.

```
git clone https://github.com/emmanuelekopimo/sem-pdf.git
cd sem-pdf
npm install

# Postgres (in the dev container)
service postgresql start
sudo -u postgres createdb sempdf
sudo -u postgres createdb sempdf_test

npm run db:migrate      # apply SQL migrations in drizzle/
npm run db:seed         # 2 users, 44 PDFs, 75 saved searches (about 5 s)
npm run dev             # http://localhost:3000
```

Variable	Default	Purpose
DATABASE_URL	postgres://postgres:postgres@localhost:5432/sempdf	Main database
SESSION_SECRET	development value	Signs session cookies. Required in production.
SEMPDF_TODAY	real date	Pin "today" (YYYY-MM-DD) for demos and tests
TEST_DATABASE_URL	.../sempdf_test	Used by Vitest and Playwright

The file `.npmrc` sets `onnxruntime-node-install=skip`: the CPU binaries already ship inside the npm package, and this stops the install script from trying to download optional CUDA files.

Scripts

<code>npm run dev / build / start</code>	Develop, build, and production start (migrate, seed if empty, serve)
<code>npm run db:generate</code>	Write a new SQL migration after editing the schema
<code>npm run db:reset</code>	Wipe and re-seed the database
<code>npm test</code>	Unit and integration tests (Vitest)
<code>npm run test:e2e</code>	Playwright, desktop and mobile
<code>npm run docs:pdf</code>	Rebuild this PDF (run <code>npm run build</code> first)

Testing

Suite	Tests	What it covers
Unit (Vitest)	59	Chunking, header stripping, keyword matching, cosine ranking and per-document caps, match labels, relative dates and history groups, SEMPDF_TODAY parsing, Zod schemas, cache freshness, JWT signing and tamper checks.
Integration (Vitest + real Postgres + real model)	15	Account creation and bcrypt login, PDF ingest with 384 dimension vectors, scanned and fake PDFs, semantic search for paraphrased questions, saved results cache and refresh after upload or delete, empty results, user scoping for documents, files and history, the full demo seed, and the health check.
End to end (Playwright)	13 (12 desktop, 1 mobile)	Redirects when signed out, inline form errors, demo login, sign up, semantic search and opening the matched passage, cached repeat, collection filter, no results, history delete, library filters, upload validation and indexing, delete, PDF download, and a Pixel 7 mobile run with an overflow check on every tab.

Total: 87 tests, all passing. Integration tests and Playwright use the `sempdf_test` database, which is migrated automatically and reset before each run. Playwright builds the app and runs it with `next start` in production mode, with `SEMPDF_TODAY=2026-10-05`.

```
pg_isready      # the container can restart Postgres
npm test        # unit + integration
npm run test:e2e # builds, seeds sempdf_test, runs desktop and mobile
```

Deployment

SemPDF runs on Railway in the **school-projects** project as the service **sem-pdf**, deployed from the GitHub repository. Public URL: <https://sem-pdf-production.up.railway.app>

railway.json

```
build: npm run build
start: npm run db:migrate && npm run db:seed:if-empty
      && npx next start -H 0.0.0.0 -p $PORT
health: /api/health (runs "select 1" against Postgres)
```

Variables

DATABASE_URL	Built from reference variables of the project's <code>Postgres</code> service, pointing at a separate <code>sempdf</code> database on that server
SESSION_SECRET	Random 64 character hex string
NODE_ENV	production
ONNXRUNTIME_NODE_INSTALL	skip (same reason as .npmrc)

Note on the database. The project has reached Railway's limit of 10 volumes, so a new Postgres service could not be added. Instead of deleting another app's database, SemPDF uses its own database named `sempdf` on the existing `Postgres` server. `scripts/migrate.ts` creates it on first boot if it is missing. Nothing in the other app's database is touched. To move to a dedicated server later, add a Postgres service once a volume is free and point `DATABASE_URL` at it; the app will migrate and seed itself.

On first boot the start command applies migrations, sees an empty database and seeds the demo data (about 10 seconds), then starts Next.js. Later restarts skip the seed and keep uploaded PDFs and history.

5 minute presentation script

Time	What to do and say
0:00 to 0:30	The problem. "Students collect dozens of PDFs. Ctrl+F only finds exact words. If my notes say maize and I search corn, I find nothing." Show the sign in page.
0:30 to 1:00	Sign in. "The demo account is filled in." Click Sign in. Point at the stats: 42 PDFs, 157 pages, 216 passages, 73 saved searches.
1:00 to 2:00	The key demo. Search <i>best time to plant corn</i> . "The Maize Farming Guide is first. The word corn is not in any document." Point at the blue box: a keyword search would find only a few of these. Point at a "Found by meaning" badge and the match bar. Open the top result and show the highlighted passage on its page.
2:00 to 2:40	More questions. Click the chip <i>can my landlord throw me out</i> (Tenancy Law), then type <i>what should I do if someone is choking</i> (First Aid). Use the collection chips to filter.
2:40 to 3:10	Fast and saved. Run the corn search again. "Loaded saved results in a few ms." Open History: grouped by day, run counts, "Will refresh" badges, and a search with nothing found.
3:10 to 3:50	Upload. Go to Upload, choose a PDF, pick a collection, click Upload and index. Show pages and passages indexed. Search for something in it. Show the scanned PDF with "No text found" in the Library.
3:50 to 4:30	How it works. "Each passage becomes 384 numbers from a small local model, all-MiniLM-L6-v2. The question becomes 384 numbers too. Cosine similarity finds the closest passages. No data leaves the server and there is no API bill."
4:30 to 5:00	Wrap up. Switch to dark mode, resize to phone width to show the tab bar. "87 automated tests, deployed on Railway." Take questions.

Backup plan: if the network fails, open this PDF and walk through the screenshots in section 4.

Decisions made

- **Model:** all-MiniLM-L6-v2, 8-bit quantized ONNX (23 MB), committed to the repository so it works offline and on Railway without downloading anything at runtime.
- **Vectors in bytea, ranking in Node:** no pgvector needed; fast enough for this scale and testable as a pure function.
- **PDFs stored in Postgres:** avoids a file volume (the Railway project is at its volume limit).
- **Passages of about 70 words with one sentence of overlap:** short enough for precise snippets, long enough for context. Max 3 results per document, top 20 overall, minimum score 0.22.
- **Saved search cache keyed on library version:** repeat searches are instant, and any upload or delete makes them refresh automatically.
- **Collections:** 8 fixed collections chosen at upload time, used for chips and colours.
- **Theme:** YouTube colours, Roboto, pill search bar and chips, but our own layout: top tabs on desktop and a bottom tab bar on mobile, no sidebar.
- **Accounts:** sign up is open so anyone can try their own private library. Demo: demo@sempdf.app / demo1234. Second demo user: tunde@sempdf.app / tunde1234.
- **Deployment database:** a separate `sempdf` database on the project's existing Postgres server, because the project has no free volumes.